

The Batch Roofline, Worked Out by Hand

HOW ONE KNOB — BATCH SIZE — TURNS A STARVED GPU INTO A FED ONE

What this document does. Module 01 proved that a 70B decoder at batch 1 extracts $\sim 0.34\%$ of an H100's peak FLOPs: the weights cross HBM once and do almost no work. This document takes the single highest-leverage cure — raising the *batch size* — and computes, from first principles, exactly how arithmetic intensity, throughput, and per-token cost respond. We sweep $B \in \{1, 8, 32, 64\}$ for the *Nexus* fraud assistant and watch tokens/sec climb while the per-token cost falls, until the KV cache runs out of room. Every figure is reproduced by `m05_t01_batch_roofline_engine.py` (Appendix A).

The one idea. Batching does not make any single request faster. It makes the *same weight read* pay for many requests at once. Arithmetic intensity grows as $AI(B) \approx B \text{ FLOP/B}$, so the GPU climbs its roofline toward the ridge point at 295.5 FLOP/B derived in Module 01.

CONCEPTS COVERED, IN ORDER

the two fixed costs of a decode step (weights + KV); arithmetic intensity vs. batch $AI(B) \approx B$; the per-step time formula; throughput and per-token cost; the batch sweep $\{1, 8, 32, 64\}$; the KV memory wall; and the tie-back to the ridge point.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system under study	2
2	Step 1 — The two costs of a decode step	2
3	Step 2 — Arithmetic intensity grows with batch	3
4	Step 3 — Per-step time, TPOT, and throughput	3
5	Step 4 — The batch sweep	4
6	Step 5 — The KV memory wall	4
7	Step 6 — Tie-back to the ridge point	5
8	The argument, at a glance	5
8.1	Equation cheat-sheet	6
	Appendix A — Reference implementation	6

1 The system under study

We reuse Module 01’s instance and add the two quantities batching cares about: the per-token KV-cache footprint, and a fixed kernel-launch overhead per step.

Quantity	Symbol	Value here
Parameters	P	70×10^9
Precision	—	FP16 (2 bytes/value)
Hidden width	d_{model}	8192
Decoder layers	N_{layers}	80
KV heads (GQA-8)	H_{kv}	8
Head dimension	d_{head}	128
HBM bandwidth (H100 SXM)	B_{HBM}	3.35×10^{12} B/s
FP16 tensor-core peak	F_{peak}	9.9×10^{14} FLOP/s
Context per request	T_{ctx}	2000 tokens
Launch overhead / step	t_{launch}	0.30 ms

Decimal units throughout (1 GB = 10^9 B). The weight footprint is unchanged from Module 01: $W_{\text{bytes}} = P \cdot 2 = 140$ GB.

Intuition

A decode *step* advances every in-flight request by exactly one token. To do that the GPU must read (a) all the weights, once, and (b) the KV cache of every active request, once. Cost (a) is *fixed* no matter how many requests share the step; cost (b) *grows* with the batch. Batching is the art of spreading the giant fixed cost (a) over as many requests as cost (b) will allow.

2 Step 1 — The two costs of a decode step

A batched decode step moves two kinds of bytes across HBM.

The KV cost per token, step by step

With GQA-8 the cache stores, per token per layer, H_{kv} key vectors and H_{kv} value vectors, each of width d_{head} , in FP16:

$$\text{kv}_{\text{tok}} = \underbrace{2}_{K, V} \times H_{\text{kv}} \times d_{\text{head}} \times N_{\text{layers}} \times 2 \text{ B} = 2 \times 8 \times 128 \times 80 \times 2 = 327680 \text{ B} = 320 \text{ KB}.$$

A request holding $T_{\text{ctx}} = 2000$ tokens therefore carries $2000 \times 320 \text{ KB} = 0.655 \text{ GB}$ of KV that must be re-read every step.

So at batch B the total bytes a step streams from HBM are

$$\text{bytes}(B) = \underbrace{W_{\text{bytes}}}_{140 \text{ GB, fixed}} + \underbrace{B \cdot T_{\text{ctx}} \cdot \text{kv}_{\text{tok}}}_{\text{grows with } B}.$$

Watch out

GQA is doing heavy lifting here. A multi-head model with 64 KV heads instead of 8 would carry $8\times$ the KV: 2.56 MB/token, and the per-request 0.655 GB would balloon to 5.2 GB. The KV wall in Step 5 would arrive $8\times$ sooner. GQA is what makes large batches affordable at all.

3 Step 2 — Arithmetic intensity grows with batch

Module 01 showed decode at batch 1 has arithmetic intensity $\text{AI} = 1 \text{ FLOP/B}$: one weight matrix is read ($d^2 \cdot 2$ bytes) and used for a single vector.

At batch B the *same* matrix read is reused by B activation vectors. The bytes are unchanged; the FLOPs multiply by B .

Decode arithmetic intensity vs. batch size (FP16)

$$\text{AI}(B) = \frac{2B d^2}{d^2 \cdot D_{\text{bytes}}} = \frac{2B}{D_{\text{bytes}}} \stackrel{D=2}{=} B \quad [\text{FLOP/B}].$$

The achievable compute follows the roofline, $\min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$, and the ridge point from Module 01 is

$$\text{ridge} = \frac{F_{\text{peak}}}{B_{\text{HBM}}} = \frac{9.9 \times 10^{14}}{3.35 \times 10^{12}} = 295.5 \text{ FLOP/B}.$$

Since $\text{AI}(B) = B$ for FP16, the GPU crosses the ridge exactly at $B = 296$. Every batch in our sweep (1 to 64) sits well below that — the pure memory-bound, linear-scaling region.

Why batching is the highest-leverage knob

$\text{AI}(B) = B$ is the whole game. Each unit of batch buys one more FLOP/B of intensity, dragging the GPU up its memory roof toward the ridge. Below $B = 296$ you are in the region where *throughput scales almost linearly with batch* at near-constant per-request latency. That is free money until the KV cache fills.

4 Step 3 — Per-step time, TPOT, and throughput

Decode is memory-bound, so step time is “bytes moved $\div$ bandwidth,” plus the fixed launch overhead.

Per-step time for a batch

$$t_{\text{step}}(B) = \frac{W_{\text{bytes}} + B \cdot T_{\text{ctx}} \cdot \text{kv}_{\text{tok}}}{B_{\text{HBM}}} \times 1000 + t_{\text{launch}} \quad [\text{ms}].$$

$$\text{TPOT}_{\text{req}} = t_{\text{step}}, \quad \text{throughput} = \frac{B}{t_{\text{step}}/1000} \quad [\text{tok/s}], \quad \text{cost/tok} = \frac{t_{\text{step}}}{B}.$$

Each request receives exactly one token per step, so its time-per-output-token (TPOT) *equals* the step time. The throughput is B tokens per step. The amortized per-token cost is the step time divided by B — and *that* is the number that collapses as we batch.

Mini-example — batch 1 vs. batch 8, the operation in isolation

At $B = 1$: bytes = 140 GB + 0.655 GB = 140.66 GB, so $t_{\text{step}} = 140.66 \times 10^9 / 3.35 \times 10^{12} \times 1000 + 0.30 = 42.29$ ms; throughput = $1/0.04229 = 23.6$ tok/s.

At $B = 8$: bytes = 140 + 5.24 = 145.24 GB, $t_{\text{step}} = 43.66$ ms; throughput = $8/0.04366 = 183.3$ tok/s. The step got 3% slower but served $8\times$ the requests: $7.8\times$ more throughput.

✓

5 Step 4 — The batch sweep

Plugging $B \in \{1, 8, 32, 64\}$ into the Step 3 formulas:

B	KV (GB)	t_{step} (ms)	TPOT (ms)	throughput (tok/s)	cost/tok (ms)	% peak
1	0.66	42.29	42.29	23.6	42.287	0.34%
8	5.24	43.66	43.66	183.3	5.457	2.71%
32	20.97	48.35	48.35	661.8	1.511	10.83%
64	41.94	54.61	54.61	1171.9	0.853	21.66%

Read the columns as a story. From $B = 1$ to $B = 64$ the per-request latency (TPOT) rose only from 42.3 to 54.6 ms (+29%), but throughput rose from 23.6 to 1172 tok/s (**49.7** $\times$), and the amortized cost per token *fell* from 42.3 to 0.85 ms — a **49.6** $\times$ reduction. The GPU climbed from 0.34% to 21.66% of peak FLOPs.

Intuition

Why does TPOT barely move while throughput explodes? Because the 140 GB weight read dominates every step and is *shared*. Adding the 8th request only adds its own 0.655 GB of KV — a 0.5% bump to the bytes moved — but it adds a whole extra token of output. You pay almost nothing per added request and collect a full token in return. That asymmetry is the entire economic case for batching.

The headline: 49 $\times$ more output, near-flat latency

At batch 64, the same H100-second that produced 23.6 tokens at batch 1 now produces 1172. Per-request latency grew 29%; per-token *cost* fell $\sim 50\times$. This is why batching is not a tuning detail — it is the difference between a viable serving business and a loss leader.

6 Step 5 — The KV memory wall

Throughput would keep rising with B forever if memory were free. It is not. The KV cache for B requests must *fit in HBM* alongside the weights. Once it does not, the scheduler must evict requests — the batch cannot grow further.

Where the wall sits

Suppose, after the 140 GB of weights and working overhead, an illustrative 60 GB of HBM remains as the KV pool. Each request at $T_{\text{ctx}} = 2000$ holds 0.655 GB. The largest batch the pool admits is

$$B_{\text{max}}^{\text{KV}} = \frac{\text{KV pool}}{T_{\text{ctx}} \cdot \text{kv}_{\text{tok}}} = \frac{60 \times 10^9}{2000 \times 327680} = 91.6 \approx 91 \text{ requests.}$$

So on this configuration the throughput curve climbs cleanly through $B = 64$, has a little more room to ~ 91 , and then *stops* — not because of compute, but because there is nowhere to put the 92nd request’s KV. The wall arrives long before the compute ridge at $B = 296$.

Watch out

Two different ceilings limit batch size, and you hit whichever comes first: **(1) the compute ridge** at $B = 296$, where $AI = 295.5$ and further batching stops helping throughput; and **(2) the KV wall** at $B \approx 91$ here, where memory runs out. For long contexts the KV wall dominates — double T_{ctx} to 4000 and $B_{\text{max}}^{\text{KV}}$ halves to ~ 45 . Capacity planning is mostly about managing the KV wall, which is why Topic 4 (scheduling) spends its time on KV-pool pressure.

7 Step 6 — Tie-back to the ridge point

Module 01 left us with one number: the ridge at 295.5 FLOP/B, the intensity at which an H100’s memory roof and compute roof cross. Batching is the lever that walks the workload along that roofline.

B	$AI = B$	vs. ridge 295.5	Regime
1	1	$\ll$ ridge	deeply memory-bound (Module 01’s floor)
64	64	$<$ ridge	memory-bound, linear-scaling region
296	296	$=$ ridge	balanced — throughput saturates here
512	512	$>$ ridge	compute-bound — TPOT rises, no throughput gain

The decode workload *starts* at the far-left of the roofline ($AI = 1$, the Module 01 bottleneck) and batching marches it rightward, one FLOP/B per unit of B , toward the ridge at 296. Our sweep covered $B \leq 64$, i.e. $AI \leq 64$ — still $4.6\times$ below the ridge, every doubling still nearly doubling throughput. The KV wall (Step 5) is what actually stops us first.

Batching, in one sentence

Batching converts the Module 01 bottleneck (one starving request at $AI = 1$, 0.34% of peak) into a fed pipeline by raising arithmetic intensity as $AI(B) = B$, climbing the roofline toward the ridge at 295.5 — stopping at whichever comes first: the ridge ($B=296$) or the KV memory wall ($B \approx 91$ here).

8 The argument, at a glance

#	Step	Result
1	Two costs	weights (140 GB, fixed) + KV (320 KB/tok, grows)
2	Intensity	$AI(B) = 2B/D = B$; ridge at 295.5 $\Rightarrow$ cross at $B=296$
3	Per-step	$t_{\text{step}} = (W + B T_{\text{ctx}} \text{ kv}) / B_{\text{HBM}} \cdot 10^3 + t_{\text{launch}}$
4	Sweep	$B:1 \rightarrow 64$ gives $49.7\times$ throughput, 29% TPOT rise
5	KV wall	$B_{\text{max}}^{\text{KV}} = \text{pool} / (T_{\text{ctx}} \text{ kv}) \approx 91$
6	Ridge	climb $AI = 1 \rightarrow 64 \dots$ toward ridge 296 ✓

8.1 Equation cheat-sheet

KV per token	$\text{kv}_{\text{tok}} = 2 H_{\text{kv}} d_{\text{head}} N_{\text{layers}} D_{\text{bytes}}$
Arithmetic intensity	$\text{AI}(B) = 2B/D_{\text{bytes}} = B \text{ (FP16)}$
Per-step time	$t_{\text{step}} = (W_{\text{bytes}} + B T_{\text{ctx}} \text{kv}_{\text{tok}}) / B_{\text{HBM}} \cdot 10^3 + t_{\text{launch}}$
TPOT / throughput	$\text{TPOT} = t_{\text{step}}; \text{ tok/s} = B / (t_{\text{step}}/1000)$
Per-token cost	t_{step}/B
Ridge crossing	$\text{AI}(B) = \text{ridge} \Rightarrow B = F_{\text{peak}}/B_{\text{HBM}}$
KV wall	$B_{\text{max}}^{\text{KV}} = \text{KV pool} / (T_{\text{ctx}} \text{kv}_{\text{tok}})$

Appendix A — Reference implementation

Every number above is produced by `m05_t01_batch_roofline_engine.py`. Change B , T_{ctx} , or the KV-pool budget to explore other operating points.

```

1  """
2  Reference implementation for: Module 05 - Topic 1
3  "The Batch Roofline - how batching turns a starved GPU into a fed one."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Convention: 1 GB = 1e9 bytes, 1 TB/s = 1e12 B/s (decimal, the convention
9  vendors use for HBM bandwidth and model "size in GB").
10 """
11
12 # -----
13 # 0. The system under study: one 70B decoder on one H100 SXM.
14 # -----
15 P = 70e9          # parameters (Llama-3-70B class)
16 bytes_fp16 = 2    # bytes per parameter / per stored value at FP16
17 d_model = 8192    # hidden width
18 n_layers = 80
19 n_kv_heads = 8    # GQA-8: 8 key/value heads
20 head_dim = 128    # 8192 / 64 query heads = 128
21 B_hbm = 3.35e12   # H100 SXM HBM3 peak bandwidth, bytes/s
22 F_peak = 990e12   # H100 FP16 tensor-core peak, FLOP/s
23 launch_ms = 0.30  # fixed per-step kernel-launch overhead (CUDA graphs)
24 context = 2000    # tokens of context (KV) held per request
25
26 # -----
27 # 1. The two fixed costs per decode step.
28 # -----
29 W_bytes = P * bytes_fp16
30 # KV cache bytes per token = 2 (K and V) * n_kv_heads * head_dim * n_layers * bytes
31 kv_bytes_per_token = 2 * n_kv_heads * head_dim * n_layers * bytes_fp16
32 print("== 1. Fixed costs ==")
33 print(f"W_bytes = {W_bytes:.3e} B = {W_bytes/1e9:.0f} GB")
34 print(f"kv_bytes_per_token = 2*{n_kv_heads}*{head_dim}*{n_layers}*{bytes_fp16}")
35 print(f"    f = {kv_bytes_per_token} B = {kv_bytes_per_token/1024:.0f} KB/token")
36
37 # -----
38 # 2. Arithmetic intensity grows linearly with batch.
39 #   Weights read once; reused by B activation vectors. AI(B) ~ B FLOP/byte.
40 # -----
41 def AI(B): return (2 * B) / bytes_fp16    # = B for FP16
42 ridge = F_peak / B_hbm
43 print("\n== 2. Arithmetic intensity vs batch ==")

```

```

44 print(f"AI(B) = 2B / {bytes_fp16} = B FLOP/byte    (ridge = {ridge:.1f} FLOP/B)")
45
46 # -----
47 # 3. Per-step time, TPOT, and throughput sweep over batch size.
48 #   step_ms = (W_bytes + B*context*kv_per_tok) / B_hbm * 1000 + launch
49 # -----
50 def step_ms(B):
51     bytes_moved = W_bytes + B * context * kv_bytes_per_token
52     return bytes_moved / B_hbm * 1e3 + launch_ms
53
54 def kv_pool_gb(B):
55     return B * context * kv_bytes_per_token / 1e9
56
57 print("\n== 3. Batch sweep (context = %d tokens) == " % context)
58 print(f"{'B':>4} {'KV GB':>8} {'step_ms':>9} {'TPOT_ms':>9} "
59       f"{'tok/s':>10} {'ms/tok':>9} {'AI':>6} {'%peak':>7}")
60 for B in [1, 8, 32, 64]:
61     s = step_ms(B)
62     tp = B / (s / 1e3)          # throughput tokens/sec (whole batch)
63     ms_per_tok = s / B         # amortized per-token cost
64     ach = min(AI(B) * B_hbm, F_peak)
65     util = 100 * ach / F_peak
66     print(f"{'B':>4} {'kv_pool_gb(B)':>8.2f} {'s:>9.2f} {'s:>9.2f} "
67           f"{'tp:>10.1f} {'ms_per_tok:>9.3f} {'AI(B)':>6.0f} {'util:>6.2f}%")
68
69 # -----
70 # 4. Where does the KV memory wall sit? KV pool budget on one H100 (80 GB).
71 #   Usable KV pool after 70 GB weights (TP=1 won't fit; assume sharded /
72 #   illustrative 60 GB KV budget). Show max batch before KV exhausts.
73 # -----
74 kv_pool_bytes = 60e9          # illustrative KV budget (bytes) after weights+overhead
75 B_max_kv = kv_pool_bytes / (context * kv_bytes_per_token)
76 print("\n== 4. KV memory wall ==")
77 print(f"kv_pool      = {kv_pool_bytes/1e9:.0f} GB")
78 print(f"per-request KV = {context*kv_bytes_per_token/1e9:.3f} GB ({context} tokens)")
79 print(f"B_max (KV-bound) = pool / per-req-KV = {B_max_kv:.1f} requests")
80
81 # -----
82 # 5. Tie back to the ridge point (Module 01): AI(B) = ridge => compute-bound.
83 # -----
84 B_ridge = ridge              # since AI(B) = B for FP16
85 print("\n== 5. Ridge crossing ==")
86 print(f"AI(B)=B, so AI=ridge at B = {B_ridge:.0f}")
87 print(f"For B < {B_ridge:.0f}: memory-bound, throughput scales ~linearly with B.")
88 print(f"For B > {B_ridge:.0f}: compute-bound, TPOT rises, throughput saturates.")
89 print(f"All swept batches (1..64) sit well below {B_ridge:.0f}: pure scaling region.")

```

— END OF DOCUMENT —